

Agent Memory Platform — Ideal Answer (Idea Level)

Practice date: 2026-10-03. Question: long-horizon memory infrastructure for AI agents running days to weeks. See `agent-memory-diagram.png` — the picture carries the structure; this page carries the ideas.

The one-sentence architecture

A tenant-scoped agent platform where a **plan document plus an agent graph** is the unit of memory, retrieval is **vector search followed by graph traversal**, and everything durable passes through **async queues with a validation gate** — because agents don't need instant answers, the whole system can be asynchronous.

The ideas that matter

1. Agents are not stateless; the platform remembers for them.

Every agent run starts by creating a plan document: what the task is, how it decomposes, which agents are involved. Agents themselves are nodes in a per-tenant graph; edges carry dependencies and weights (whose output feeds whose input). When a new task arrives — from a user or from an agent waking up — the first question is always: can it leverage work existing agents already did, or is this genuinely new?

2. Memory has three temperatures.

Working memory (hot, minutes) holds the live run state. The plan document and agent graph hold the orchestration state. The long-term store (weeks) holds embedded task completions in a vector index plus structured facts (preferences, decisions). A daily compaction job turns weeks of turns into key points and archives finished work — without it, the store grows unbounded and retrieval drowns.

3. Retrieval is search, then walk, then rank.

Embed the current task, vector-search for similar prior work, walk the graph for dependencies and agent state, then rank candidates by similarity x recency x importance and take the top-k that fit the token budget. Across parallel threads, reserve budget per thread so one topic can't starve the others.

4. Routing doesn't need a model.

Deciding which agents to invoke is rules-based — user tier, usage patterns, SLA urgency — not a per-turn LLM call. A routing model at 12k turns/sec would be a fleet of its own.

5. Writes are queued, gated, and checkpointed.

Write-back goes through a queue into a sanitizer: validate before anything becomes durable. Suspicious memories are quarantined, not deleted. The platform checkpoints last-known-good state so a poisoned interaction can be rolled back rather than corrupting all future runs.

6. Contradictions resolve by recency with provenance.

When two memories disagree (email vs. Slack preference), the newer one wins and the older is tombstoned but retained for audit. Asking the user is reserved for high-stakes ambiguity, queued rather than blocking the run.

7. Deletion is a first-class citizen, not a cron afterthought.

A GDPR deletion service spans every tier with tombstones plus hard delete. Per-tenant namespaces guarantee one tenant's vectors can never surface in another's retrieval.

8. You know it works by measuring retrieval, not vibes.

Precision/recall against similar completed tasks as ground truth: did retrieval surface what the successful run actually used? And the behavioral proxy that needs no labels: follow-up questions on a topic mean retrieval missed something.

What was deliberately not built

- No per-turn routing LLM (cost).
- No synchronous read path (agents tolerate notification-style latency; async everywhere).
- No user-visible token-budget mechanics (internal constraint, handled silently).
- No physical deletion on every write path (tombstones + lazy GC, except where the deletion SLA demands otherwise).